



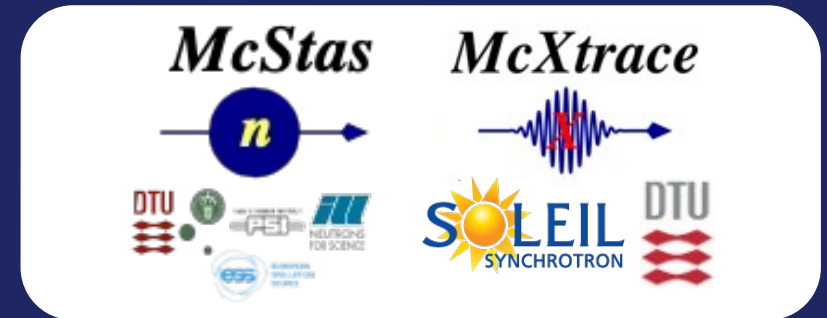
A COMMON INTRODUCTION

McStas & McXtrace

Monte Carlo ray-tracing for neutron and X-ray instrumentation

McStas — neutrons

McXtrace — X-rays



PaNRAID School · supported by PEPR DIADEM



Who's behind this school



PaNRAID is organised with support from the France 2030 PEPR DIADEM programme, training researchers to combine McStas and McXtrace ray-tracing simulations with AI methods.

PARTNERS & SUPPORTING ORGANISATIONS





What we'll cover

1

Shared origins & philosophy

One codebase family, two physics engines

2

The Monte Carlo ray-tracing idea

Why rays, weights, and randomness

3

Anatomy of a simulation

Instruments, components, and the ray state

4

Under the hood

DSL, code generation, and coordinate placement

5

Neutrons vs. X-rays

Where the two probes really differ

6

Community, use & getting started

License, ecosystem, and next steps



One family, two probes

McXtrace was built on proven McStas technology — same architecture, adapted for X-rays.

Commonality: same domain-specific language, same component-based architecture, same open, community-driven development model.

1998

2000s

2009

Today

McStas is born

Grows into a general tool

McXtrace forks off

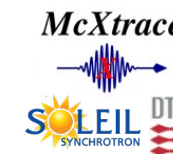
Parallel development

K. Nielsen & K. Lefmann, Risø.
Originally for triple-axis
spectrometer simulation.

Adopted by DTU Physics, ILL,
PSI, University of Copenhagen,
ESS DMSC. GPL open source.

“Powered by McStas
technology.” Re-targeted at
X-ray optics & beamlines.

Shared kernel philosophy,
separate physics & component
libraries, active communities.

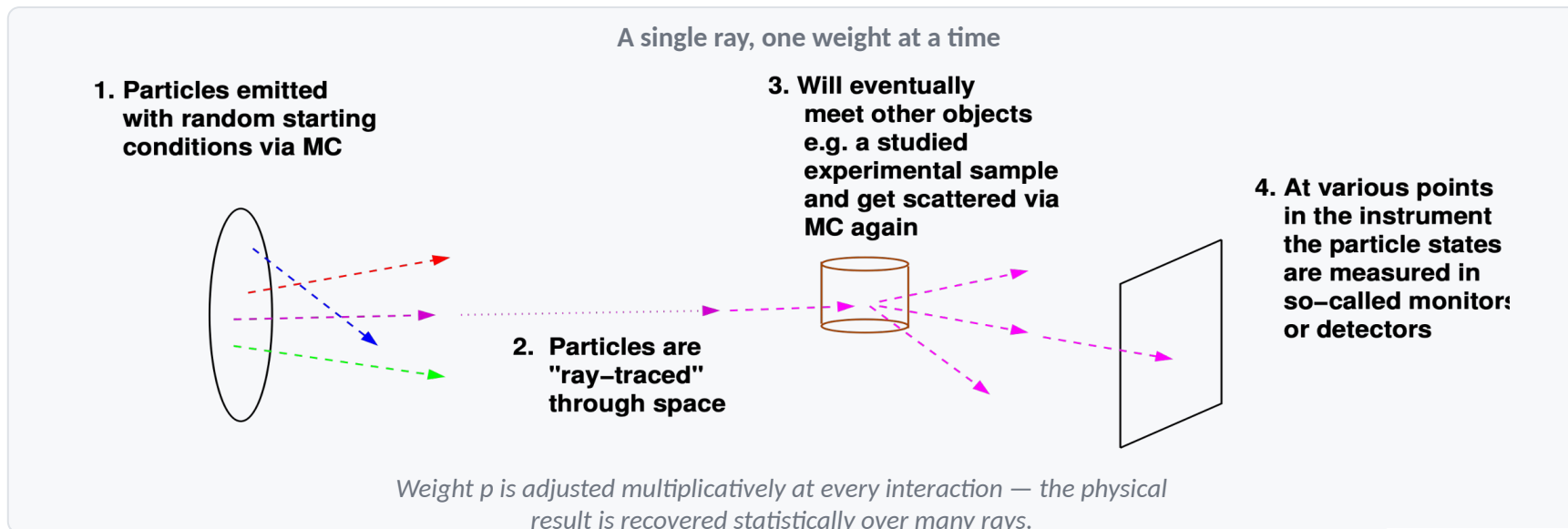




Monte Carlo ray-tracing, in a nutshell

A statistical method born at Los Alamos in the 1940s (*von Neumann, Ulam, Metropolis*) for solving particle-transport problems by numerical experiment.

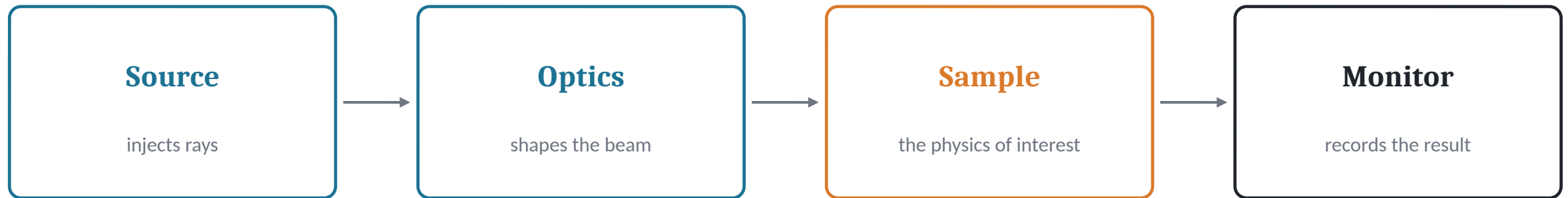
- Both codes trace many independent rays (neutrons or photons) one at a time through the instrument.
- Propagation in free space is deterministic — straight lines (plus gravity, for slow neutrons).
- Where physics is probabilistic — scattering, absorption, reflection — outcomes are Monte Carlo sampled.
- Each ray carries a statistical weight rather than representing one real particle, so rare but important events can be sampled efficiently.





Anatomy of a simulation

An instrument is an ordered chain of components in a shared lab coordinate system. Rays flow through it, changing state at each stop.



Instrument: defines the lab coordinate system. **Components:** devices/features available in the instrument, each with its own function. **Rays:** passed from one component to the next, changing state along the way.

This chain — and the DSL that describes it — is identical in McStas and McXtrace.



What travels through the beamline: the ray

Neutron ray · McStas

r	position (x, y, z)
v	velocity vector
t	time
s	spin / polarisation
p	statistical weight

Photon ray · McXtrace

r	position (x, y, z)
k	wave vector
φ, E	phase & polarisation (E-field)
t	time
p	statistical weight

Same idea, different carrier: both rays are position + time + statistical weight. The neutron adds mass, velocity and spin; the photon adds a wave vector, phase and field polarisation.



Component classes

Every instrument, in either code, is built from the same four component classes.

Class	McStas (neutrons)	McXtrace (X-rays)
Sources	Reactor / spallation moderator	Undulator, wiggler, bending magnet
Optics	Guides, mirrors, choppers, collimators	Mirrors, monochromator crystals, multilayers, slits
Samples	Powders, crystals, liquids, magnetism	Powders, crystals, multilayers, absorbing phantoms
Monitors	PSD / histograms / event lists	PSD / histograms / event lists (same concept)

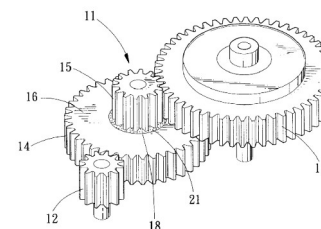
Common to all components: they set, manipulate, or measure the state of the ray — and each is written as a self-contained, reusable code module.



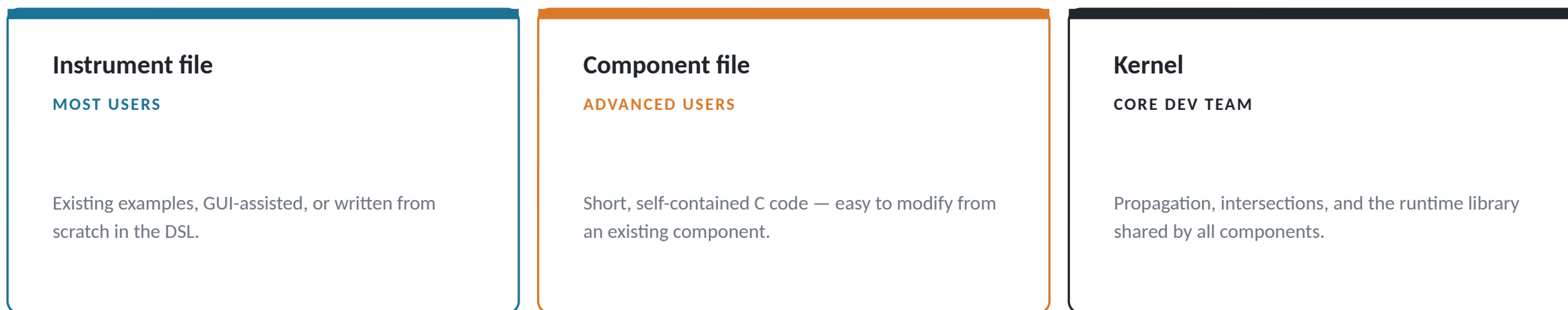
From instrument file to running simulation



Same toolchain in both codes — the DSL grammar and code generator are shared.



Three levels of code, three audiences



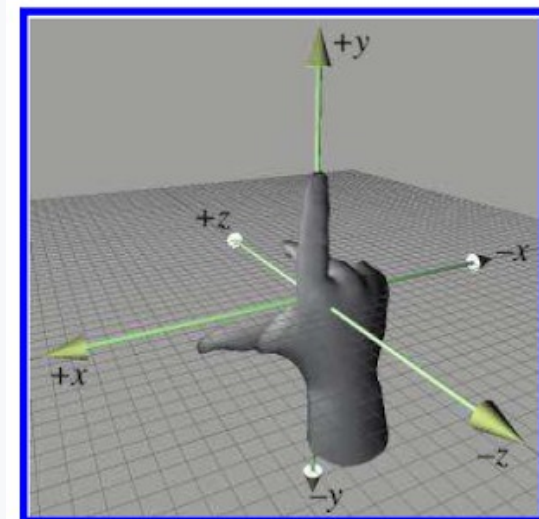
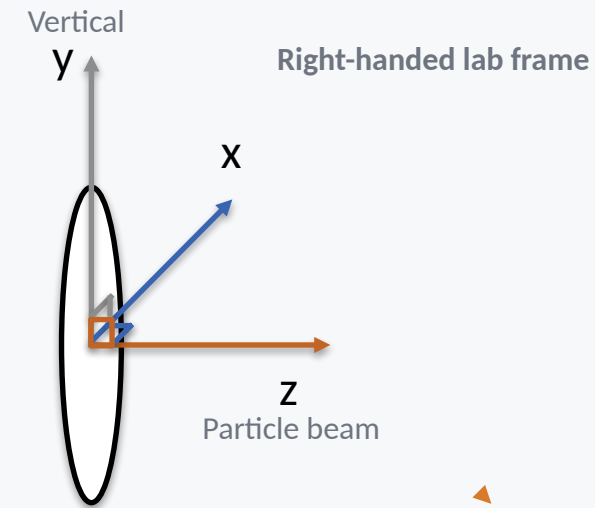


Placing components in space

Every component is placed with the same two statements, in both codes:

```
AT (x, y, z) RELATIVE <ref>
ROTATED (rx, ry, rz) RELATIVE <ref>
```

- <ref> can be ABSOLUTE (the lab frame), another component's name, or PREVIOUS.
- z runs along the beam, y is vertical, x completes a right-handed frame.
- Component authors choose whatever local origin is physically natural — the kernel handles the coordinate transforms between them.
- AT and ROTATED can be fully decoupled, referring to different RELATIVE <ref>





Neutrons vs. X-rays: key differences at a glance

	Neutrons • McStas	X-rays • McXtrace
Probe	Neutral particle — mass, spin	Massless electromagnetic wave
Typical facility	Reactor or spallation source	Synchrotron or free-electron laser
Energy scale	meV (thermal / cold)	keV (hard X-ray)
Dominant interaction	Nuclear & magnetic scattering; deep penetration	Electronic scattering; surface-sensitive, strongly absorbed

What stays the same: the ray-tracing method, the DSL, the component architecture, and the statistical-weight formalism are identical — only the physics inside the components differs.



Open, portable, community-built

GPL

Open-source license
for both packages

3

Platforms: Linux,
macOS, Windows

175+

Components in the
shared-style libraries

30–40%

Of new components
come from users

- Runs everywhere from a laptop to HPC clusters and GPUs (OpenACC) — the same generated C code, recompiled per target.
- New components automatically fit the framework once written in structured, standard C.
- Contributions require at least one test instrument and a documented header — then they join the shared contrib/ library.
- Permanent staff at DTU Physics maintain the core; a wider network of institutes and students keeps both libraries growing.



Who builds & maintains them

A shared core team at DTU Physics, plus partner institutes around each package.

McStas partner network

- DTU Physics (core development)
- European Spallation Source (ESS DMSC)
- University of Copenhagen
- Institut Laue-Langevin (ILL), Grenoble
- Paul Scherrer Institute (PSI)

McXtrace partner network

- Synchrotron SOLEIL
- DTU Physics (core development)
- DTU Space
- University of Copenhagen

Further both codes benefit from the greater network of users across a plethora of use-cases and applications.

Several developers work across both projects — shared infrastructure means fixes and features often benefit both codes at once.



What are they used for?

1

Instrument design

Planning and optimising new beamlines before they're built

2

Construction support

Verifying components and layouts during commissioning

3

Virtual experiments

Letting prospective users try an experiment before travelling to a facility

4

Data analysis

Comparing measured data against simulated instrument response

5

Teaching

Training students and new users in instrumentation & scattering methods

Both communities use their code across the full lifecycle — from first sketch to published data.



A shared, interoperable ecosystem

1 MCPL

The Monte Carlo Particle List format lets McStas, McXtrace and other transport codes (MCNP, Geant4, PHITS, ...) exchange rays and couple simulations together.

2 Graphical interface

A common Python-based GUI supports building, running and browsing results for instruments in either code.

3 Plotting tools

A shared plotting toolchain (HTML/D3, pyqtgraph, matplotlib backends) visualises monitor output from both packages.

4 Packaging

Conda-forge distributions and cross-platform installers keep both codes easy to install on Linux, macOS and Windows.

Shared interfaces, tools and utilities



The image displays a collage of screenshots related to the McStas software suite:

- McStas GUI (mogui-py):** Shows the main interface with buttons for Docs, Run, Plot, Open, and Edit. It includes a message box with copyright information and a component library.
- 3D Visualization:** A central window showing a 3D model of a neutron beamline with various components and a red beam path.
- Jupyter Notebook:** A window showing Python code for configuring the instrument and generating plots. The code includes comments and parameters for the instrument configuration.
- Plots:** Several plots are shown, including a 2D intensity map, a 1D intensity profile, and a 3D intensity map.



Where to go next

McStas

WEBSITE

mcstas.org

MAILING LIST

mcstas-users@mcstas.org

INSTALL

[conda-forge](#) · [cross-platform installers](#)

SOURCE

[GitHub](#) — [McStasMcXtrace/McCode](#)

McXtrace

WEBSITE

mcxtrace.org

MAILING LIST

mcxtrace-users@mcxtrace.org

INSTALL

[conda-forge](#) · [cross-platform installers](#)

SOURCE

[GitHub](#) — [McStasMcXtrace/McCode](#)

**micromamba env create -n panraid mcstas mcxtrace -y
(apt install mcstas-suite-python-ng mcxtrace-suite-python-ng)**

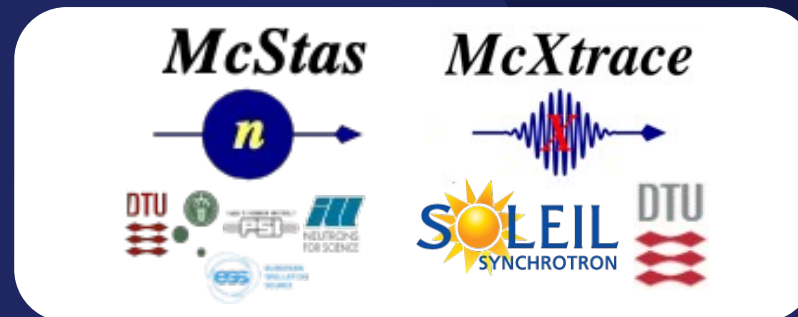


Thank you

Questions?

mcstas.org

mcxtrace.org



SUPPORTED BY



Peter Willendrup · pkwi@fysik.dtu.dk · DTU Physics